
USRT Heuristic

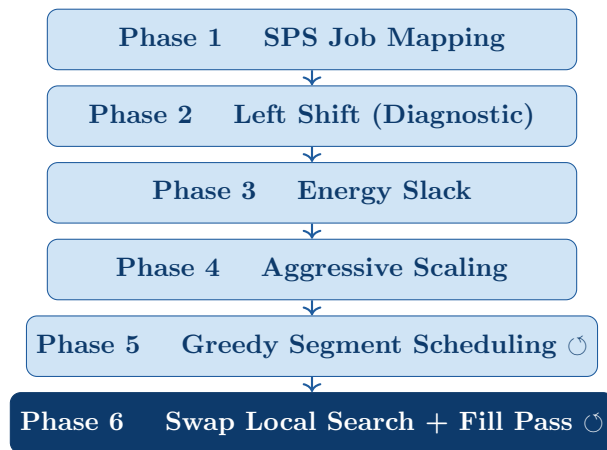
Utility-aware Energy-Constrained Real-Time Scheduling

Algorithm Design Document

Problem: Map N_{tsk} periodic real-time tasks across m processors.

Goal: Maximise utility from optional segment execution.

Constraints: EDF deadlines · Global energy budget B · No migration.



Contents

1	Introduction & Problem Statement	2
2	System Model	2
2.1	Processor Model	2
2.2	Task & Job Model	2
2.3	Energy Model	2
2.4	Utility Model	3
3	Decision Variables & Constraints (ILP Reference)	3
4	Algorithm Overview	4
5	Correctness Guarantees	9
6	Complexity Analysis	9
7	Function Reference	10
	References	10

Introduction & Problem Statement

The **USRT** (Utility-aware Scheduling of Real-Time tasks) problem schedules N_{tsk} periodic real-time tasks on N_{prc} homogeneous processors under a global energy budget B , *maximising* the total weighted utility gained from executing optional task segments. Execution is **preemptive EDF**, jobs are **mapped at the job level**, and all task periods are **harmonic**.

Constraint	Description
No deadline misses	Mandatory segment of every job must finish by its deadline under EDF.
Energy budget	Total energy across all jobs in the hyper-period $\leq B$.
No migration	Each job executes on exactly one processor; no mid-execution movement.
Single frequency	A job cannot change its execution frequency once it has started.
EDF ordering	Each processor services jobs in earliest-deadline-first order.

System Model

Processor Model

A set $\mathcal{P}$ of N_{prc} *homogeneous* processors. Each processor P_x executes at a discrete, normalised frequency set $\mathcal{F} = \{f_1 < f_2 < \dots < f_{\text{max}}\}$ where $f_{\text{max}} = 1.0$.

Task & Job Model

A set $\mathcal{T} = \{T_i\}$ of N_{tsk} periodic tasks with *harmonic* periods. The j^{th} job of task T_i is denoted $T_{i,j}$.

Symbol	Meaning
p_i	Period of task T_i
$r_{i,j} = (j - 1) \cdot p_i$	Release (arrival) time of job $T_{i,j}$
$d_{i,j} = j \cdot p_i$	Deadline of job $T_{i,j}$
$h = \text{lcm}\{p_i\}$	Hyper-period; $N_i^{\text{job}} = h/p_i$ jobs of task T_i
$G_{i,0}$	Mandatory segment of T_i
$G_{i,k} \ (k \geq 1)$	k^{th} optional segment (cumulative: executing $G_{i,k}$ implies $G_{i,0}, \dots, G_{i,k-1}$ done)
$e_{i,k}$	Execution time to complete up to the k^{th} segment at f_{max}
N_i^{seg}	Number of optional segments for task T_i

Energy Model

The **effective execution time** for job $T_{i,j}$ running up to segment k at frequency f_z is:

Equation 1 — Effective Execution Time

$$e_{i,j,k,z}^{\text{eff}} = \frac{1}{f_z} \sum_{q=0}^k e_{i,q} \quad (1)$$

The **energy consumed** by that job is:

Equation 2 — Energy Model

$$E_{i,j,k,z} = \alpha \cdot e_{i,j,k,z}^{\text{eff}} + \beta \cdot f_z^2 \cdot \sum_{q=0}^k e_{i,q} \quad (2)$$

where α is the static (leakage) power coefficient and β is the dynamic power coefficient. Default values: $\alpha = 1.0$, $\beta = 0.5$.

Energy minimum at $f_{\max}$: Setting $\partial E / \partial f_z = 0$ in (2):

$$f_{\text{opt}} = \left(\frac{\alpha}{2\beta} \right)^{1/3} = \left(\frac{1}{1} \right)^{1/3} = 1.0 = f_{\max}$$

Therefore, for $\alpha = 1, \beta = 0.5$, *decreasing* frequency *always increases* total energy. Phase 4 (Aggressive Scaling) is consequently a structural no-op for these default coefficients, but is implemented generically for arbitrary α, β .

Utility Model

Utility is **linear** in optional execution time. Let u_i denote the utility per unit of optional execution for task T_i . The utility of job $T_{i,j}$ executing up to segment k is:

Equation 3 — Job Utility

$$U_{i,j}^{\text{job}} = u_i \times \sum_{q=1}^k e_{i,q} \quad (k \geq 1) \quad (3)$$

The **total utility objective** (to be maximised) is:

$$\Phi = \sum_i \sum_j U_{i,j}^{\text{job}}$$

Decision Variables & Constraints (ILP Reference)

The heuristic approximates the ILP v1 formulation. The primary decision variable is:

ILP v1 Decision Variable

$$V(i, j, k, x, z) \in \{0, 1\} \iff \text{job } T_{i,j} \text{ executes on } P_x \text{ at frequency } f_z \text{ up to segment } k$$

The heuristic maintains equivalent state via `seg_k[(i,j)]` and `freq_idx[(i,j)]`.

Label	Mathematical Form	Description
C1	$\sum_{k,x,z} V(i,j,k,x,z) = 1 \forall i,j$	Each job gets exactly one (k,x,z) assignment.
C2	$\text{DBF}(t_1, t_2, x) \leq t_2 - t_1 \forall x, t_1, t_2$	Demand bound feasibility per processor per window. Checked by <code>check_all_timing()</code> .
C3	$E^{\text{total}} \leq B$	Total energy of all jobs $\leq$ budget B .
Obj	$\max \Phi = \sum_{i,j,k,z} u_i \cdot (\text{cum}[i][k] - \text{cum}[i][0]) \cdot Y(i,j,k,z)$	Maximise utility from optional segments.

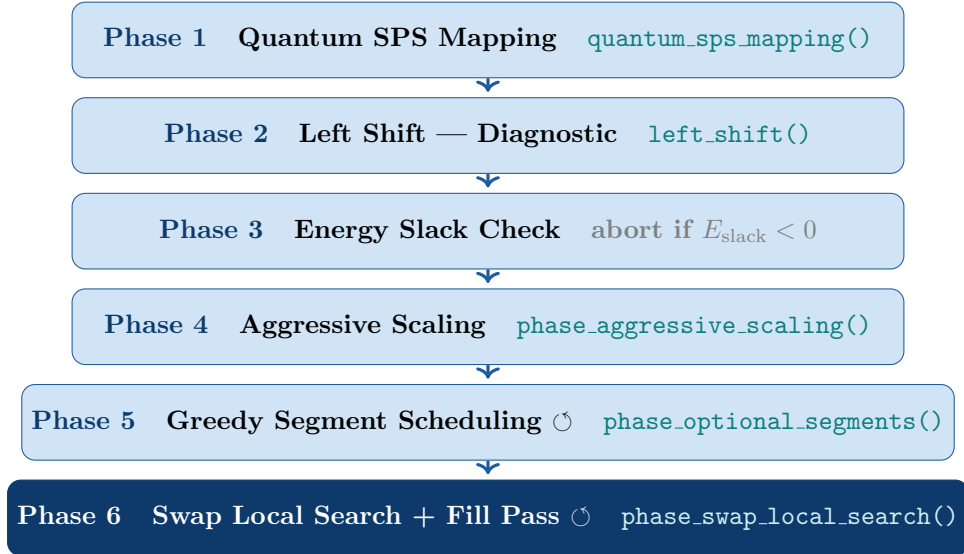
The **Demand Bound Function** for processor P_x over window $[t_1, t_2]$ is:

DBF Definition

$$\text{DBF}(t_1, t_2, x) = \sum_{T_{i,j} \in S(t_1, t_2, x)} e_{i,j,k,z}^{\text{eff}} \quad (4)$$

where $S(t_1, t_2, x) = \{T_{i,j} : P_{i,j}^* = x, r_{i,j} \geq t_1, d_{i,j} \leq t_2\}$.

Algorithm Overview



Phase 1 — Quantum SPS Mapping

Functions: `quantum_sps_mapping()` `run_dps()` `run_sps()` `_check_dbf_mandatory()`
`_repair()`

5.1 Motivation

The **Sum Partial Solutions (SPS)** algorithm [1] solves the makespan-minimisation problem for independent jobs on identical parallel machines in $\mathcal{O}(n \log n)$. It is adapted here for job-to-processor mapping using *utilisation* $u_i = e_m/p_i$ as the load metric — directly compatible with the preemptive EDF feasibility condition.

5.2 Quantum Structure

The hyper-period $h = \text{lcm}\{p_i\}$ is divided into quanta of length $\delta = \text{gcd}\{p_i\}$. At each quantum $[q, q + \delta]$:

Mandatory $d_{i,j} = q + \delta$: must be mapped this quantum or the deadline is missed.

Optional $d_{i,j} > q + \delta$: can be deferred; sorted by deadline ascending (most urgent first).

5.3 SPS Algorithm

Algorithm 1 DPS — Determining Partial Solutions

Input: Jobs with loads $\{(i, j), e_m/p_i\}$ sorted descending

Output: Almost-balanced p -family of partial solutions $\mathcal{S}$

```

1: Initialise  $\mathcal{S} \leftarrow \{\text{singleton}(\text{first job})\}$ 
2: for all remaining jobs  $(i, j)$  with load  $v$  do
3:   Find  $S_r \in \mathcal{S}$  with maximum gap  $\Delta_r = \text{loads}[0] - \text{loads}[-1]$ 
4:   if  $v \leq \Delta_r$  then
5:     Insert  $(i, j)$  into least-loaded machine of  $S_r$ ; re-sort
6:   else
7:      $\mathcal{S} \leftarrow \mathcal{S} \cup \{\text{singleton}(i, j)\}$  ▷ new partial solution
return  $\mathcal{S}$ 

```

Algorithm 2 SPS — Combining Partial Solutions

Input: Almost-balanced p -family $\mathcal{S}$

Output: Single final mapping S_{final}

```

1: while  $|\mathcal{S}| > 1$  do
2:   Select  $S_l, S_k \in \mathcal{S}$  with two largest gaps
3:    $S_{\text{new}} \leftarrow \text{Ord}(S_l \bowtie S_k)$  ▷ machine  $j$  gets  $S_l[j] \cup S_k[m-1-j]$ , then re-sort
4:    $\mathcal{S} \leftarrow (\mathcal{S} \setminus \{S_l, S_k\}) \cup \{S_{\text{new}}\}$ 
return  $\mathcal{S}[0]$ 

```

5.4 Almost-Balanced Family

DPS produces an *almost-balanced p -family* $\mathcal{S} = \{S_1, \dots, S_p\}$ satisfying:

- (a) $t_1^r \geq \dots \geq t_m^r \quad \forall r$ (loads sorted descending per partial solution)
- (b) $t_1^r \leq 2t_m^r \quad \forall r \in \{1, \dots, p-1\}$ (max $\leq 2 \times$ min)

By Proposition 2.6 of [1], the *gap does not increase* under the ordered combination operator, ensuring the final mapping is well-balanced.

5.5 Worst-Case Performance Ratio

Theorem 4.1 (Chiaselotti et al. 2010)

$$\frac{C_{\max}}{C_{\max}^*} \leq 1 + \frac{m-1}{m \cdot p}$$

where p is the number of initial partial solutions produced by DPS. A larger p gives a tighter bound; $p = 1$ implies an optimal solution.

Phase 2 — Left Shift — Diagnostic

Functions: `left_shift()`

Purpose: Estimate idle time available per job. Run **once** after Phase 1. This is a non-preemptive proxy for diagnostic reporting only — *not* the actual preemptive EDF schedule. The results are *not* recomputed inside the optimisation loop.

Procedure: For each processor, sort jobs in EDF order then simulate:

Left Shift

$$\begin{aligned} \text{start}_{i,j} &= \max(\text{proc_avail}, r_{i,j}) \\ \text{finish}_{i,j} &= \text{start}_{i,j} + e_{i,j,k,z}^{\text{eff}} \\ \text{ls_slack}_{i,j} &= d_{i,j} - \text{finish}_{i,j} \end{aligned}$$

Left-shift invariant: Time slack exists *only in the future* (a job cannot start before $r_{i,j}$). Negative ls_slack does not imply infeasibility under preemptive EDF.

Phase 3 — Energy Slack

Compute remaining energy budget after running all jobs at $f_{\max}$ with mandatory segments only ($k = 0$):

Equation 4 — Energy Slack

$$E^{\text{slack}} = B_{\text{budget}} - \sum_{i,j} E(i, k=0, z=f_{\max}) \quad (5)$$

If $E^{\text{slack}} < 0$: mandatory-only execution already exceeds the budget. The algorithm reports **infeasibility** and returns.

Phase 4 — Aggressive Scaling

Functions: `phase_aggressive_scaling()`

Iteratively try to **decrease frequency** of each job by one level. Commit only if both conditions hold:

1. $E_{\text{new}} < E_{\text{cur}}$ (energy strictly decreases)
2. Timing feasible across all DBF windows (`check_all_timing()`)

For $\alpha = 1, \beta = 0.5$: decreasing frequency always increases energy $\Rightarrow$ **no-op**. Generically correct for other α, β configurations.

Phase 5 — Greedy Optional Segment Scheduling

Functions: `phase_optional_segments()` `min_slack_for_job()` `window_slack()`
`check_all_timing()`

Job Ordering

Jobs are sorted in **stable descending order of u_i** . Ties broken by task index then job index for determinism. The job with the highest u_i is denoted j^* .

DBF Window Slack — Timing Gate

For job (i_s, j_s) on processor x , the minimum available DBF slack across all *containing* windows is:

Equations 5–6 — DBF Window Slack

$$\text{window_slack}(x, t_1, t_2) = (t_2 - t_1) - \text{DBF}(t_1, t_2, x) \quad (6)$$

$$\text{min_slack}(i_s, j_s, x) = \min_{\substack{t_1 \leq r_{i_s, j_s} \\ t_2 \geq d_{i_s, j_s}}} \text{window_slack}(x, t_1, t_2) \quad (7)$$

Key theorem: If $\text{min_slack}(i_s, j_s, x) \geq \Delta t_{\text{add}}$, then adding Δt_{add} more execution to job (i_s, j_s) keeps *all* containing windows feasible. No separate DBF call is needed for Case (i) — the slack check *is* the proof of feasibility.

Three Cases (1 Segment per j^* per Pass)

The outer `while` loop repeats until a full pass over all jobs adds nothing.

Case green!70!black: Case (i) — Without Impacting Others

Required:

$$\text{min_slack}(j^*) \geq \underbrace{\frac{\text{cum}[i_s][k+1] - \text{cum}[i_s][k]}{f_z}}_{\Delta t_{\text{add}}} \quad \text{AND} \quad E^{\text{slack}} \geq \underbrace{E(i_s, k+1, z) - E(i_s, k, z)}_{\Delta E_{\text{add}}}$$

Action: `seg_k[(i_s, j_s)] += 1`. Update E^{slack} . No extra DBF call needed.

Case midblue: Case (ii.A) — Increase Frequency of j^*

Try $z^* \rightarrow z^* + 1$ (higher frequency $\Rightarrow$ shorter execution, less energy in our model).
 Net time change and net energy change when moving from (k, z) to $(k+1, z+1)$:

$$\Delta t = e^{\text{eff}}(i_s, k+1, z+1) - e^{\text{eff}}(i_s, k, z)$$

$$\Delta e = E(i_s, k+1, z+1) - E(i_s, k, z)$$

Pre-check: $\text{min_slack} - \Delta t \geq 0$ **AND** $E^{\text{slack}} - \Delta e \geq 0$.

Safety: `check_all_timing()` before committing.

Case teal: Case (ii.B) — Decrease Frequency of Others on Same Processor

Candidates: other jobs on same processor with $u_i \leq u_{j^*}$ and frequency $> f_{\min}$.

Sorted: lowest u_i first (sacrifice least valuable).

For each candidate (i_2, j_2) with current frequency z_2 :

$$\text{energy saved} = E(i_2, k_2, z_2) - E(i_2, k_2, z_2 - 1)$$

Phase 6 — Best-First Pairwise Segment Swap

Functions: `phase_swap_local_search()` `min_slack_for_job()` `check_all_timing()`
`phase_optional_segments()`

6.1 Motivation: Why Greedy Gets Stuck

Phase 5 converges to a **local optimum** — it cannot undo earlier decisions. A typical failure:

Example: Job A ($u = 3.0$, large $e_{o,k}$) added first by Phase 5. DBF window is now full. Job B ($u = 2.9$, small $e_{o,k}$) is blocked. But removing one of A's segments frees enough window for B.
 Net utility: $+u_B \cdot e_{B,next} - u_A \cdot e_{A,last} > 0 \Rightarrow$ profitable swap that Phase 5 *cannot discover*.

6.2 Swap Definition

A **pairwise segment swap** between a *giver* (i_1, j_1) and a *receiver* (i_2, j_2) :

Equation 7 — Segment Swap

Remove last segment of giver: $\text{seg_k}[(i_1, j_1)] := k_1 - 1$

Add next segment to receiver: $\text{seg_k}[(i_2, j_2)] := k_2 + 1$

Net utility: $\Delta U = u_{i_2}(\text{cum}[i_2][k_2 + 1] - \text{cum}[i_2][k_2]) - u_{i_1}(\text{cum}[i_1][k_1] - \text{cum}[i_1][k_1 - 1])$

6.3 Feasibility Conditions for a Valid Swap

- (1) $\Delta U > 0$ (net utility gain is strictly positive)
- (2) $\Delta E_{\text{freed}} - \Delta E_{\text{cost}} \leq E^{\text{slack}}$ (global energy balance)
- (3) $\text{min_slack}(\text{receiver, with giver's seg removed}) \geq \Delta t_{\text{add}}(\text{receiver})$
- (4) `check_all_timing()` global full check before final commit

The receiver's timing check (condition 3) uses the **modified state** with the giver's segment already tentatively removed. If giver and receiver share a processor, the window slack increases — correctly capturing the cross-job interaction.

6.4 Cross-Processor Swaps

Cross-processor swaps are **fully supported**: energy is a global resource, so removing a segment on P_0 can fund a segment on P_1 . Only the per-processor DBF timing is checked independently for each side.

6.5 Best-First Selection

In each iteration, **all valid (giver, receiver) pairs** are evaluated. The pair with maximum ΔU is executed. Complexity: $\mathcal{O}(N_{\text{jobs}}^2 \times N_{\text{windows}})$ per swap iteration — practical for typical instances.

6.6 Algorithm

Algorithm 3 Phase 6 — Best-First Pairwise Segment Swap

```

1: while true do
2:   best  $\leftarrow \emptyset$ 
3:   for all giver  $(i_1, j_1)$  with  $\text{seg\_k}[(i_1, j_1)] > 0$  do
4:     Tentatively:  $\text{seg\_k}[(i_1, j_1)] -= 1$ 
5:     for all receiver  $(i_2, j_2)$  with  $\text{seg\_k}[(i_2, j_2)] < N_{i_2}^{\text{seg}}$  do
6:       Compute  $\Delta U$ , check energy, check min_slack ▷ with giver's seg removed
7:       if  $\Delta U > 0$  and all conditions satisfied then
8:         if  $\Delta U > \text{best}.\Delta U$  then best  $\leftarrow (i_1, j_1, i_2, j_2, \Delta U)$ 
9:         Restore  $\text{seg\_k}[(i_1, j_1)] += 1$ 
10:    if best =  $\emptyset$  then break
11:    Execute best swap; run check_all_timing(); update  $E^{\text{slack}}$ 

```

Correctness Guarantees

All USRT constraints are strictly enforced at every phase. No constraint is ever relaxed to gain utility.

Constraint	How Enforced	Function(s)
No deadline miss (C2)	Phase 1: utilisation ≤ 1.0 per processor (Liu & Layland). Phase 5/6: $\text{min_slack} \geq \Delta t$ before every segment addition.	<code>min_slack_for_job()</code> <code>check_all_timing()</code>
Energy budget (C3)	E^{slack} tracked incrementally. Every addition checks $E^{\text{slack}} \geq \Delta E$. Phase 3 aborts if mandatory cost exceeds budget.	<code>total_energy()</code>
No migration	Processor assignment from Phase 1 is permanent. Phases 5/6 never move jobs between processors.	<code>quantum_sps_mapping()</code>
Single frequency	<code>freq_idx[(i,j)]</code> changed at most once per Phase 5 iteration; reverted if DBF fails.	<code>phase_optional_segments()</code>
Mandatory execution	All jobs initialised at $k = 0$ (mandatory only). Optional segments are additive. k never decreases in Phase 5.	<code>Initialisation in run_heuristic()</code>
EDF ordering	DBF feasibility $\Rightarrow$ EDF schedulability. Confirmed at each step.	<code>check_all_timing()</code>

Complexity Analysis

Phase	Time Complexity	Notes
Phase 1 (SPS)	$\mathcal{O}(n \log n)$ per quantum	DPS sort dominates. Repair adds $\mathcal{O}(n^2)$ worst-case.
Phase 2 (Left Shift)	$\mathcal{O}(n \log n)$ once	EDF sort per processor.
Phase 4 (Scaling)	$\mathcal{O}(n \cdot N_{\text{freq}} \cdot N_{\text{win}})$	Iterates until no change; no-op for $\alpha = 1, \beta = 0.5$.
Phase 5 (Greedy)	$\mathcal{O}(\text{passes} \cdot n \cdot N_{\text{win}})$	$N_{\text{win}} = \mathcal{A} \times \mathcal{D} $ per processor. Typically ≤ 20 passes.
Phase 6 (Swap)	$\mathcal{O}(\text{swaps} \cdot n^2 \cdot N_{\text{win}})$	Each swap triggers a Phase 5 fill pass. Practical: $n \lesssim 80$, swaps $\lesssim 10$.

Function Reference

Function Name	Phase	Description
<code>run_heuristic()</code>	Top	Orchestrates all phases. Builds <code>proc_jobs</code> , <code>freq_idx</code> , <code>seg_k</code> state.
<code>quantum_sps_mapping()</code>	1	Per-quantum SPS loop: mandatory/optional split, utilisation pre-filter, DPS+SPS, DBF verify.
<code>run_dps()</code>	1	Determining Partial Solutions: greedy job insertion into almost-balanced partial solutions.
<code>run_sps()</code>	1	Combines partial solutions iteratively using ordered combination until one remains.
<code>_check_dbf_mandatory()</code>	1	Checks DBF feasibility using e_m only (mandatory baseline at $f_{\max}$).
<code>_repair()</code>	1	Greedy repair: moves heaviest offending job to least-loaded processor.
<code>left_shift()</code>	2	Non-preemptive EDF simulation; computes $ls_slack_{i,j} = \text{deadline} - \text{finish}$.
<code>phase_aggressive_scaling()</code>	4	Iteratively decreases frequency if $E_{\text{new}} < E_{\text{cur}}$ and timing holds.
<code>phase_optional_segments()</code>	5	Outer while loop; 1 segment/ j^* /pass; Cases (i), (ii.A), (ii.B).
<code>window_slack()</code>	5,6	DBF slack = $(t_2 - t_1) - \sum e_{\text{eff}}$ of jobs in $S(t_1, t_2, x)$.
<code>min_slack_for_job()</code>	5,6	Min window_slack over all windows containing (i_s, j_s) . Sufficient timing gate.
<code>check_all_timing()</code>	5,6	Full DBF check across all processors and all (t_1, t_2) pairs.
<code>phase_swap_local_search()</code>	6	Best-first pairwise swap: search all (giver,receiver); execute best; fill pass.
<code>total_energy()</code>	util	$\sum_{i,j} E(i, \text{seg_k}[(i, j)], \text{freq_idx}[(i, j)])$.
<code>total_utility()</code>	util	$\sum_{i,j} u_i \cdot (\text{cum}[i][\text{seg_k}[(i, j)]] - \text{cum}[i][0])$.
<code>energy_val(cum_k, fz)</code>	util	$\alpha(c/f) + \beta f^2 c$ (Eq. 2).
<code>e_eff_val(cum_k, fz)</code>	util	c/f (Eq. 1, effective execution time).
<code>build_cum(tasks)</code>	util	$\text{cum}[i][k] = e_{i,0} + \dots + e_{i,k}$ (cumulative exec times at $f_{\max}$).

References

References

- [1] G. Chiaselotti, M. I. Gualtieri, P. Pietramala, “Minimizing the Makespan in Nonpreemptive Parallel Machine Scheduling Problem,” *Journal of Mathematical Modelling and Algorithms*, vol. 9, pp. 39–51, 2010.
- [2] USRT Problem Statement — Utility-aware Energy-Constrained Real-Time Scheduling, *Segmented_Workload_Scheduling_Shared_v2*.
- [3] C. L. Liu, J. W. Layland, “Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment,” *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973.
- [4] N. Kumar, J. Mayank, A. Mondal, “Reliability Aware Energy Optimized Scheduling of Non-preemptive Periodic Real-Time Tasks on Heterogeneous Multiprocessor System,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 31, no. 4, pp. 871–885, 2019.